Scholarship Intelligence Crawler

Technical documentation | Version 1.1.0

1. Architecture

- Application: Streamlit displays saved records, filters, source evidence and confidence scores. Original source-check dates are preserved. Full evidence files prepare in the background.
- Background worker: Crawl now starts a live update while users continue browsing. Daily crawling runs at 09:00 IST while the application is running. Terminal commands use the same workflow.
- Discovery layer: config/seeds.json defines starting websites, official domains and discovery rules. Relevant pages and supporting PDFs are grouped by scholarship.
- Source layer: each downloaded source is saved with its URL, retrieval time and content hash. HTML is prepared into readable blocks; PDFs keep physical page locations.
- Processing layer: Python rules extract a common record structure. Evidence checks validate claims. Confidence rules calculate the score and verification status.
- Storage layer: SQLite stores facts, sources, snapshots, evidence, crawl runs and accepted changes. Transactions and foreign keys protect updates and relationships.
- Presentation layer: the dashboard reads stored results. Unsupported claims remain withheld or marked for review.
- Separate replay: official-notice replay uses an isolated SQLite database. Its records and history tables store replay cases and transitions without changing main scholarship records.

2. Technology choices

- Python 3.11: workflow, extraction, verification and scoring. No language-model API key or paid service is required.
- Requests: HTTP downloads and conditional checks. Scrapy / Parsel: crawl policies, link discovery and HTML selection.
- pypdf: text-PDF extraction with page references. Pydantic: checks the common data structure.
- SQLite: records, source evidence and history. Streamlit: dashboard, search, filters and crawl controls.
- uv: locked dependency installation. unittest: automated regression tests.

3. Discovery methodology

- Starting sources: FFE, IIT Delhi, NSP, Chevening India and HDFC ECSS. They cover trust, university, government, international and corporate CSR funding.
- Follow relevant links and catalogue entries beyond the starting URLs. Source rules identify individual opportunities and supporting material.
- Group content by scholarship. Exclude awards and other non-scholarship programme types from the main view. Apply crawl budgets, request spacing and access rules.

4. Extraction methodology

- Read HTML headings, paragraphs, lists, tables and links; read text PDFs with physical page numbers.
- Extract name, provider, official source, benefits, eligibility, application link, opening/closing dates, documents, selection and renewal.
- Keep source quotations and locations for important claims. Missing information stays unspecified; ambiguous information is flagged or withheld.

5. Verification methodology

- Confirm that the source belongs to the configured official provider and relates to the scholarship.
- Check source hashes and the identity of the extraction bundle. Match quotations against the saved source text.
- Check referenced HTML blocks or PDF pages, and whether each passage supports the field in context.
- Check completeness of relevant supporting wording, unresolved conflicts and required source material.
- Official-source identity confirmation is separate from whole-record verification.

6. Confidence-score methodology

- Each component is scored from 0 to 100. Python calculates the score from evidence checks.

Confidence = 0.30 x A + 0.50 x F + 0.20 x C

- A - official-source authenticity (30%): 100 when the configured official-ownership check passes; otherwise 50.
- F - field support (50%): 100 x supported applicable fields / total applicable fields. With no applicable fields, F is 0.
- Supported fields and fields explicitly checked as not applicable count as supported. Unsupported or unresolved applicable fields reduce support.
- Fields confirmed as not specified in the inspected material are excluded from the fraction. They do not earn support points. Missing core fields still block verification.
- C - dated offering context (20%): 100 when required sources are fresh and the application-window or allowed current-cycle checks pass; otherwise 0.
- Context can be a supported opening/closing window, or a supported current cycle with a supported closing date or explicitly supported no-application route.
- Required source retrievals must be within 30 days and pass freshness checks. A recent download alone does not prove that applications are open.
- VERIFIED requires confidence of at least 95 and all critical checks to pass. Name, provider, official source, benefits and eligibility must be supported.
- The application route must be supported or checked as not applicable. Conflicts, incomplete required supporting material or failed freshness/context checks prevent verification.
- If a critical check fails, the final score is capped at 94 and the record remains REVIEW_REQUIRED. No artificial score floor or quota bonus is added.

7. Anti-hallucination approach

- Extract facts from source content rather than generating missing information.
- Preserve original bytes, quotations, URLs, hashes and source locations so important claims can be traced.
- Do not infer an application cycle from a filename or unrelated date. Do not turn conditional benefits into an unsupported fixed amount.
- Keep conflicting or incomplete claims unresolved. Unknown dates do not imply an open application window.
- Keep automated test fixtures separate from real application records.

8. Change and expiry detection

- Revisit listing pages and known sources on every live crawl. Use ETag or Last-Modified headers when supported.
- After a valid HTTP 304 response and integrity check, reuse saved bytes. Otherwise, download the source and compare its hash.
- Reuse unchanged extraction only while artifacts, configuration and processing rules still match. Process new or changed content again.
- Accepted changes preserve old/new values, detection times and evidence references. Unresolved updates remain review proposals.
- Evaluate expiry separately from verification, using supported application dates. Failed source access requires re-verification rather than automatic expiry.

9. Historical official-notice replay

- Objective: demonstrate two deadline changes and two expired examples using genuine official source material, without waiting for providers to publish new updates.
- Meaning of historical replay: the system fetches older and later published notices and processes them in order. It compares the earlier deadline with the later deadline, keeping both values and their evidence.
- For expiry, it uses a real published application window. It evaluates that window at an earlier date when applications were open, then at a later date after closing.
- The notices and dates are real. The sequence is replayed now; it does not claim that production crawls witnessed those historical changes as they happened.
- NMMSS and Sacred Heart College CSSS renewal instructions provide deadline revisions. The CSSS example concerns institutional instructions, not an independently established national policy revision.
- Chevening India and DAAD HSK provide the historical expiry windows. Replay output is stored separately from the main database.

10. Results

- Retained live validation run completed on 15 October 2026, starting with an empty database.
- 296 scholarship records across five source categories; official-source identity confirmed for all 296.
- 4 whole-record verified scholarships with confidence >=95; 292 records requiring review.
- 119 active or expiring-soon records; 9 expired records; average confidence 83.1.
- Zero language-model API calls. The final incremental-workflow release check passed 157 automated regression tests.
- Official replay on 15 October 2026 at 22:38 IST: 2 deadline changes, 2 expired examples, all reported evidence checks passed, no failures and no production rows changed.
- Both changed deadlines moved from 30 September to 31 October 2026 and were unexpired on the replay check date.
- These are retained check results. Source updates, freshness and evaluation dates can change later results.